Research on Cross-Layer Link Fault Root Cause Localization Algorithm for Microservice Architecture Integrating Deep Reinforcement Learning

Zhengdong Wu*, Haixiang Fu, Lin Wang, Xinqiang Wang, Shujun Zhang, Yuzhe Xiang, Weiyi Ge
Information System Integration Branch Company, Nari Technology Co.Ltd, Nanjing, China
*wuzhengdong@sgepri.sgcc.com.cn, fuhaixiang@sgepri.sgcc.com.cn, wanglin7@sgepri.sgcc.com.cn, wangxinqiang@sgepri.sgcc.com.cn, zhangshujun@sgepri.sgcc.com.cn, xiangyuzhe@sgepri.sgcc.com.cn, geweiyi@sgepri.sgcc.com.cn

2026 IEEE 6th International Conference on Electronic Technology, Communication and Information (ICETCI) | 979-8-3315-5907-6/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICETCI68937.2026.11610700

***Abstract*—This paper proposes a cross-layer link fault root cause localization algorithm for microservice architecture integrating deep reinforcement learning to address the problems of cross-layer fault propagation concealment, heterogeneous feature semantics, and the tendency of traditional methods to get trapped in local optima. First, a fault propagation weighted directed graph (FPWDG) model is constructed. Through cross-layer data collection and feature engineering, a unified representation of multi-level fault features is achieved, and an incremental update mechanism is adopted to ensure the model's dynamics. Second, fault root cause localization is transformed into a Markov decision process, and an improved DQN algorithm (Attention-DQN) integrating an attention mechanism is designed. Global optimal root cause search is achieved through a multi-objective weighted reward function and a pruning mechanism. A cloud-native simulation platform is built to conduct experimental verification in three scenarios: single root cause, multiple root cause, and dynamic link. Results show that the proposed algorithm achieves a localization accuracy of 96.5%, which is 7%-34% higher than mainstream algorithms. The average number of troubleshooting steps is reduced by more than 47%, and the false positive rate is as low as 4.7%. In the dynamic link scenario, the accuracy remains above 90%. This algorithm overcomes the limitations of traditional methods, providing a highly efficient root cause analysis solution for cloud-native systems, significantly improving troubleshooting efficiency and reducing operational costs.**

Keywords—Microservice architecture, cross-layer link failure, root cause analysis, deep reinforcement learning, fault propagation model, attention mechanism

I. Introduction

Microservice architecture, with its loose coupling and scalability, has become the mainstream architectural paradigm for cloud-native applications, widely deployed in core areas such as fintech, e-commerce platforms, and the industrial internet. With the surge in the number of microservices and the increasing complexity of call chains, cross-layer link failures (such as application layer interface timeouts, network layer latency jitter, and cascading failures of infrastructure layer resource bottlenecks) are frequent, exponentially increasing the difficulty of root cause analysis [1]. The core challenge of cross-layer link failures lies in the concealed and non-monotonic nature of fault propagation, and the semantic heterogeneity of fault characteristics across different layers, making it difficult for traditional analysis methods to construct global fault correlations.

Existing research on microservice fault localization mainly falls into three categories: rule-based reasoning methods rely on human experience and have weak generalization ability; machine learning-based methods require a large amount of labeled data and have poor adaptability to dynamic links; and graph neural network-based methods focus on static topology modeling and cannot capture the temporal characteristics of fault propagation [2]. Current research on cross-layer fault localization suffers from three main shortcomings: first, a lack of a unified cross-layer feature fusion mechanism; second, failure to consider the dynamic evolution characteristics of fault propagation; and third, a tendency to get trapped in local optima, making it difficult to balance accuracy and efficiency.

This research proposes a cross-layer link fault root cause localization algorithm that integrates deep reinforcement learning [3]. By constructing a dynamic fault propagation model and an intelligent search strategy, it overcomes the limitations of traditional methods. The theoretical value of this research lies in establishing a quantitative analysis framework for cross-layer fault propagation and enriching the theoretical system of microservice observability; its application value lies in improving the efficiency of fault diagnosis in cloud-native systems and reducing operational costs [4]. The core research content includes: constructing a weighted directed graph model for fault propagation to achieve unified representation and dynamic correlation of cross-layer fault features; designing a deep reinforcement learning localization algorithm that integrates attention mechanisms to achieve globally optimal root cause search; and building a simulation experimental platform to verify the algorithm's performance in multiple scenarios [5]. The technical roadmap covers cross-layer data acquisition, feature engineering, model construction, algorithm training, experimental verification, and solution implementation, forming a complete technical closed loop.

II. Construction of a Cross-Layer Link Fault Propagation Model for Microservices

A. Cross-Layer Link Data Acquisition and Feature Engineering

The cross-layer link data collection scope covers three

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 21:41:41 UTC from IEEE Xplore. Restrictions apply.

core layers: application layer, network layer, and infrastructure layer [6]. The collected metrics are divided into four categories: 1) Application layer metrics, including interface response time, error rate, and call frequency; 2) Network layer metrics, including transmission latency, packet loss rate, and bandwidth utilization; 3) Infrastructure layer metrics, including CPU utilization, memory usage, and disk I/O; and 4) Link topology metrics, including service call relationships, dependency strength, and link hop count. The data collection period is set to 100ms, and real-time collection and storage are achieved using Prometheus + Grafana.

The core of cross-layer feature fusion is to solve the semantic heterogeneity problem of features at different levels [7]. The specific steps are as follows: Semantic mapping: Discrete features (such as error types) are transformed into numerical vectors through one-hot encoding, and time-series features (such as response time series) are extracted using a sliding window to obtain statistical features such as mean, variance, and trend slope, achieving semantic unification of cross-layer features; Feature standardization: Z-Score standardization is used to eliminate dimensional differences, with the formula $x_{norm} = (x - \mu)/\sigma$, where x is the original feature value, μ is the feature mean, and σ is the feature standard deviation; Feature dimensionality reduction: Principal component analysis (PCA) is used to remove redundant features, retaining the top 90% of principal components by contribution rate, constructing a unified fault feature space $F = \{f_1, f_2, \dots, f_d\}$ with dimension $d = 32$, providing a data foundation for subsequent fault propagation modeling.

B. Construction of a Weighted Directed Graph Model for Fault Propagation

A Fault Propagation Weighted Directed Graph (FPWDG) model is proposed to achieve dynamic quantitative representation of cross-layer link fault propagation relationships. The model is formally defined as $G = (V, E, W, T)$, with the following elements defined:

— Node set V: Contains three types of nodes: service nodes, link nodes, and source-stealing nodes. Service nodes correspond to microservice instances, link nodes correspond to inter-service call links, and resource nodes correspond to infrastructure such as servers and containers [8]. Node attributes are feature vectors in a unified fault feature space.

— Edge set E: Directed edges represent the fault propagation direction, including service call edges (service node → service node), link dependency edges (service node → link node), and resource support edges (resource node → service node). The existence of edges is determined by the real-time link topology.

— Weight set W: Edge weights represent the probability of fault propagation, calculated as $w_{ij} = \mathrm{corr}(f_i, f_j) \times \alpha + \mathrm{dep}_{ij} \times (1 - \alpha)$, where $\mathrm{corr}(f_i, f_j)$ is the Pearson correlation coefficient between the feature vectors of node i and node j, reflecting the strength of feature association; dep_ij is the dependence strength of node i on node j, obtained by normalizing historical call frequency; $\alpha = 0.7$ is the feature association weight coefficient, which has been experimentally verified as the optimal configuration.

— Timestamp T: Records the model update time, realizing dynamic evolution characteristics.

The model's dynamic update mechanism adopts an incremental update strategy. When new feature data is collected, only the coefficient vectors of the changed nodes and the weight values of the associated edges are updated [9]. The update cycle is consistent with the data acquisition cycle (100 ms), avoiding the performance overhead of full updates and ensuring that the model can reflect changes in the fault propagation state in real time.

III. Design of a Cross-Layer Link Fault Root Cause Localization Algorithm Integrating Deep Reinforcement Learning

A. Core Algorithm Design Ideas

The essence of microservice cross-layer link fault root cause localization is to find the starting node of fault propagation in the FPWDG model. This process can be transformed into a dynamic path search optimization problem: starting from the fault alarm node, search for the path with the minimum cost in a weighted directed graph; the starting node of the path is the root cause of the fault [10]. Traditional localization methods use a greedy strategy for local search, which is prone to getting trapped in local optima, leading to incorrect root cause localization.

Deep Reinforcement Learning (DRL) has the ability to achieve globally optimal decisions through trial and error learning in dynamic environments, effectively solving the above problems. This study models the fault root cause localization process as a Markov Decision Process (MDP). The agent selects nodes for state transitions in the FPWDG model, accumulates rewards through interaction with the environment, and finally learns the optimal root cause search strategy [11]. The core innovation of the algorithm lies in integrating cross-layer features into the state representation, introducing an attention mechanism to strengthen the feature weights of key nodes, and using an improved DQN network to achieve policy optimization, breaking through the local optima limitation of traditional methods.

B. Design of Key Algorithm Components

1) The state of an agent consists of three parts: current node features, neighboring node features, and global topological features. Formally defined as:

$$s_t = \left[f_{\text{curr}} \oplus \mathcal{A}\left(f_{\text{neigh}} \right) \oplus g_{\text{global}} \right] \tag{1}$$

Where $f_{\text{curr}} \in \mathbb{R}^d$ is the feature vector of the current node; f_{reigh} is the feature matrix of the neighboring nodes; $\mathcal{A}(\cdot)$ is the attention mechanism used to calculate the weighted feature vector of the neighboring nodes, the weights of which are determined by the probability of fault propagation between nodes; $g_{global} \in \mathbb{R}^k$ is the global topological feature, including statistics such as the average path length of the graph and the degree distribution of nodes; $(+)$is the vector concatenation operation. The dimension of the state space is $d + d + k = 64$, ensuring the completeness and simplicity of the state representation.

2) Action Space. An action is defined as the agent selecting a neighboring node from the current node for transition, or determining the current node as the root cause and terminating the search. The action space is formally represented as:

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 21:41:41 UTC from IEEE Xplore. Restrictions apply.

$$a_t \in \left\{ v_{\text{neigh}} \cup \{ \text{stop} \} \right\} \quad (2)$$

Where v_{neigh} represents the set of neighboring nodes of the current node; stop is the termination action. The size of the action space dynamically changes with the number of neighboring nodes of the current node to ensure search flexibility.

3) Reward Function. The reward function design needs to balance localization accuracy and search efficiency, employing a multi-objective weighted reward mechanism, formally defined as:

$$r_t = \lambda_1 \cdot r_{acc} + \lambda_2 \cdot r_{eff} - \lambda_3 \cdot r_{step} \quad (3)$$

Where r_{acc} is the accuracy reward, set to 1 if the current node is the root cause, and 0 otherwise; r_{eff} is the efficiency reward, determined by the fit between the failure propagation probability of the current node and the root cause; r_{step} is the step penalty to prevent the agent from over-searching; $\lambda_1 = 0.5, \lambda_2 = 0.3, \lambda_3 = 0.2$ are the reward weight coefficients, determined through experimental optimization.

4) Network Structure. An improved DQN network with an integrated attention mechanism (Attention-DQN) is used as the policy network. The network structure consists of three layers:

— Attention Feature Extraction Layer: Inputs the feature matrix of neighboring nodes, and calculates the weighted feature vector through the attention mechanism, using the following formula:

$$\mathcal{A}(f_{\text{neigh}}) = \sum_{i=1}^{n} \alpha_i \cdot f_i, \alpha_i = \frac{\exp(w_{\text{exar},i})}{\sum_{j-1}^{n} \exp(w_{\text{eurr},j})} \quad (4)$$

Where α_i is the attention weight, calculated from the failure propagation probability $w_{curr,i}$ between the current node and its neighboring node i.

— Fully connected hidden layer: Contains two fully connected layers, using ReLU activation function to achieve non-linear mapping of state features, as shown in the formula:

$$h = \text{ReLU}(W_1 \cdot s_t + b_1), h' = \text{ReLU}(W_2 \cdot h + b_2) \quad (5)$$

Where $W_1 \in \mathbb{R}^{64\times128}, W_2 \in \mathbb{R}^{128\times64}$ are weight matrices; b_1, b_2 are bias vectors.

— Q-value output layer: Outputs the Q-value for each action, with the formula:

$$Q(s_t, a_t) = W_3 \cdot h' + b_3 \quad (6)$$

Where $W_3 \in \mathbb{R}^{64\times|A|}$ is the weight matrix, and $|A|$ is the action space size.

The training process is optimized using a target network and an experience replay mechanism. The target network parameters are updated every 1000 steps, and the experience replay pool size is set to 10^5 to ensure training stability.

C. Algorithm Execution Flow

The complete execution steps of the algorithm are as follows:

- Initialization: Construct the FPWDG model, initialize the Attention-DQN network parameters, and empty the experience replay pool; generate an initial state s_0 starting from the fault alarm node.
- Action Selection: Use an ϵgreedy strategy to select actions. When $\epsilon > \text{rand}(0,1)$, a random action is selected; otherwise, the action with the largest Q-value is selected. ϵ linearly decays from 0.9 to 0.1.
- State Transition: Execute action a_t. If the action is stop, the search terminates; otherwise, transition to a neighboring node, generate a new state s_{t+1}, and calculate the reward r_t.
- Experience Storage: Store (s_t, a_t, r_t, s_{t+1}) into the experience replay pool. When the pool is full, remove the oldest experience.
- Network Training: Randomly sample batches of experience from the experience replay pool, calculate the target Q-value $y_t = r_t + \gamma \cdot \max_a Q'(s_{t+1}, a)$, where $\gamma = 0.95$ is the discount factor, and Q' is the Q-value of the target network; minimize the loss function through gradient descent: $L = \frac{1}{N}\sum_{i=1}^{N} \left(y_i - Q(s_i, a_i)\right)^2$.
- Pruning Optimization: Introduce a pruning mechanism. When the failure propagation probability of a node is lower than the threshold $\theta = 0.1$, remove it from the search space to reduce invalid search steps and improve search efficiency.
- Result Output: When the agent executes the stop action, output the current node as the root cause of the failure, completing the localization process.

The algorithm achieves a balance between globally optimal root cause localization and efficient search by dynamically adjusting the search strategy and pruning optimization. The overall steps are clear and highly operable.

IV. Experimental Simulation and Performance Analysis

A. Experimental Environment Setup

The experiment uses a cloud-native simulation platform to build a microservice test environment. The specific configuration is as follows:

- Hardware Environment: The server consists of 4 Intel Xeon E5-2680 v4 CPUs, 128GB of RAM, and 1TB SSDs, with a network bandwidth of 10Gbps; the client consists of 10 virtual machines, configured with Intel Core i7-10700 CPUs and 16GB of RAM.
- Software Environment: The operating system is Ubuntu 20.04 LTS, the container orchestration is Kubernetes v1.25, the microservice framework is Spring Cloud, the data acquisition tool is Prometheus v2.40, the fault injection tool is Chaos Mesh v2.5, and the deep learning framework is PyTorch v1.13.
- System Deployment: An e-commerce system comprising 20 microservices is deployed, covering core functions such as user authentication, product management, order payment, and logistics tracking. Services communicate via RESTful API calls,

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 21:41:41 UTC from IEEE Xplore. Restrictions apply.

building cross-layer links encompassing the application layer, network layer, and infrastructure layer.

- Fault Injection Module: Supports injection of three types of faults: application layer (interface timeout, exception throwing), network layer (latency, packet loss), and infrastructure layer (CPU overload, memory leak). Parameters such as fault type, severity, and injection time can be configured.

B. Experimental Dataset and Evaluation Metrics

The experimental dataset was generated through fault injection, covering three scenarios: single-root cause, multiple-root cause, and dynamic link, generating a total of 10,000 fault samples [12]. Single-root cause scenarios accounted for 60% (6,000 samples), multiple-root cause scenarios accounted for 20% (2,000 samples), and dynamic link scenarios accounted for 20% (2,000 samples, with link topology changing dynamically over time). The dataset was divided into a training set (7,000 samples) and a test set (3,000 samples) in a 7:3 ratio to ensure consistent scenario distribution between the training and test sets. Core evaluation metrics include:

- Accuracy: The proportion of correctly located fault samples out of the total samples.
- Average Steps (AS): The average number of node transitions required to locate the root cause, reflecting the algorithm's search efficiency.
- Recall: In multi-root cause scenarios, the proportion of correctly identified root causes to the actual number of root causes, reflecting the algorithm's multi-root cause localization capability.
- Training Time: The time required for the algorithm to converge during training, reflecting the algorithm's engineering practicality.

C. Algorithm Selection for Comparison

Four representative fault localization algorithms were selected for comparison. The core configurations of each algorithm are as follows:

- Rule-Based Algorithm (RB): Employs manually defined fault investigation rules (e.g., "CPU utilization exceeding 90% is considered a root cause"). The rules are written by senior operations engineers, representing traditional operations methods.
- Decision Tree-Based Algorithm (DT): Uses the C4.5 decision tree algorithm, with input features being cross-layer fault features. The decision tree depth is set to 10, representing traditional machine learning methods.
- Traditional DQN Algorithm: Employs the standard DQN network, where the state representation only includes the current node's features. It lacks attention mechanisms and pruning optimization, representing a fundamental reinforcement learning method.
- Graph Neural Network (GNN) Based Algorithm: Employs a Graph Convolutional Neural Network (GCN), with inputs being the adjacency matrix and node features of the FPWDG model. The number of layers is set to 3, representing a mainstream graph learning method.

D. Experimental Results and Analysis

1) Experimental Results in Single-Root Cause Scenarios

Table I shows the comparison of localization accuracy and average number of investigation steps for each algorithm in single-root cause scenarios.

TABLE I. PERFORMANCE COMPARISON OF ALGORITHMS IN SINGLE-ROOT CAUSE SCENARIOS

Algorithm	Positioning accuracy (%)	Average number of investigation steps	Training time (min)
RB	62.3	12.5	-
DT	78.6	8.2	15
Traditional DQN	85.7	6.1	60
GNN	89.2	5.8	75
This paper's algorithm	96.5	3.2	85

As shown in Table I, the localization accuracy of the proposed algorithm reaches 96.5%, an improvement of 34.2% compared to the RB algorithm and 7.3% compared to the GNN algorithm. The average number of troubleshooting steps is only 3.2, a reduction of 47.5% compared to the traditional DQN algorithm, indicating that the attention mechanism and pruning optimization effectively improve the localization accuracy and efficiency of the algorithm.

Figure 1 shows the accuracy curves of each algorithm as a function of fault severity in a single-root cause scenario. The horizontal axis represents fault severity (levels 1-5, with level 5 being the most severe), and the vertical axis represents localization accuracy. The accuracy of the RB algorithm improves significantly with fault severity, but the accuracy is only 75.1% at level 5 fault. The proposed algorithm maintains an accuracy above 95% at all severity levels, demonstrating the best stability.

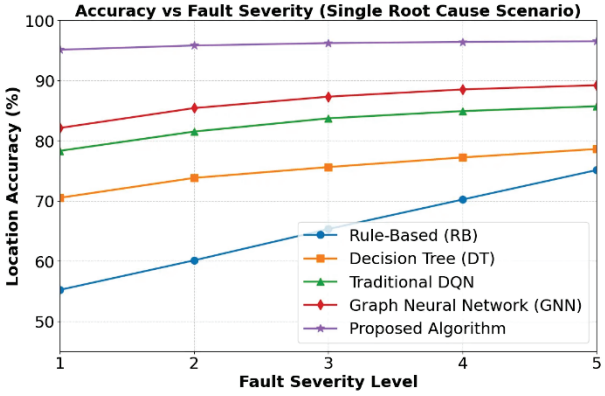


Figure 1. Accuracy variation curves with fault severity in single-root-cause scenarios.

2) Experimental results in multi-root-cause scenarios.

Table II shows the comparison of recall rate and average number of investigation steps for each algorithm in multi-root-cause scenarios. As can be seen from Table II, the recall rate of the algorithm presented in this paper reaches 92.8%, which is 20.7% higher than the traditional DQN algorithm, and the false positive rate is only 4.7%, indicating that the algorithm has excellent localization capabilities in multi-root-cause scenarios.

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 21:41:41 UTC from IEEE Xplore. Restrictions apply.

TABLE II. PERFORMANCE COMPARISON OF ALGORITHMS IN MULTI-ROOT CAUSE SCENARIOS

Algorithm	Recall rate (%)	Average number of investigation steps	False positive rate (%)
RB	45.2	18.6	32.5
DT	63.7	12.3	21.8
Traditional DQN	72.1	9.5	15.3
GNN	81.5	8.7	10.2
This paper's algorithm	92.8	5.6	4.7

Figure 2 shows the recall rate of each algorithm as a function of the number of root causes in a multi-root cause scenario. The horizontal axis represents the number of root causes (1-4), and the vertical axis represents the recall rate (%). As the number of root causes increases, the recall rate of each algorithm decreases. The algorithm in this paper still maintains a recall rate of 88.3% when there are 4 root causes, which is significantly better than other algorithms.

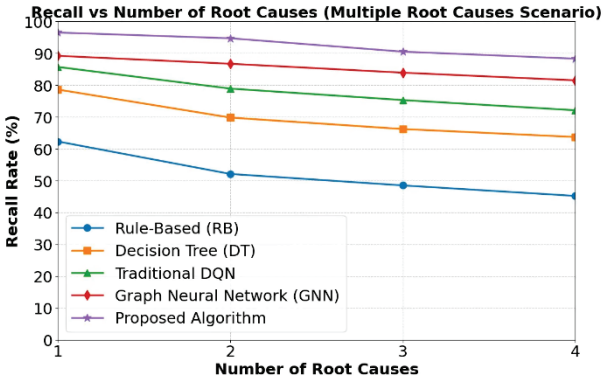


Figure 2. Recall rate as a function of root cause number in a multi-root cause scenario.

3) Experimental results in a dynamic link scenario.

In a dynamic link scenario, the link topology is updated every 10 seconds. The localization accuracy of each algorithm changes over time, as shown in Figure 3. The accuracy of traditional DQN and GNN algorithms gradually decreases over time, dropping to 72.3% and 78.5% respectively at 100 seconds. The algorithm in this paper, employing a dynamic FPWDG model, maintains an accuracy consistently above 90%, demonstrating optimal adaptability.

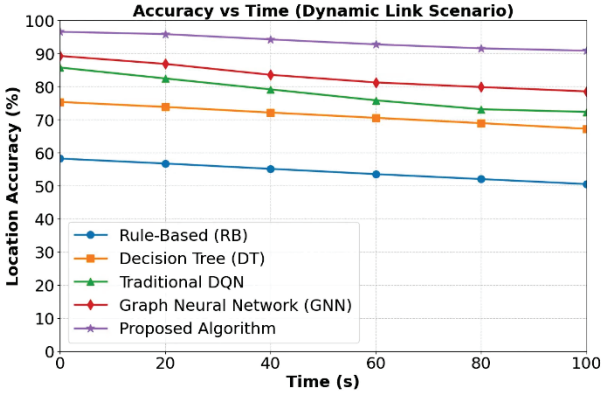


Figure 3. Accuracy over time in dynamic link scenarios.

4) Parameter Sensitivity Analysis

A sensitivity analysis was performed on the key parameters of the algorithm (attention weight coefficient α, discount factor γ, pruning threshold θ), and the results are shown in Figure 4. When α=0.7, γ=0.95, θ=0.1, the algorithm accuracy reached a peak of 96.5%; when the parameters deviated from the optimal values, the accuracy decreased slowly, indicating that the algorithm has good robustness to the parameters.

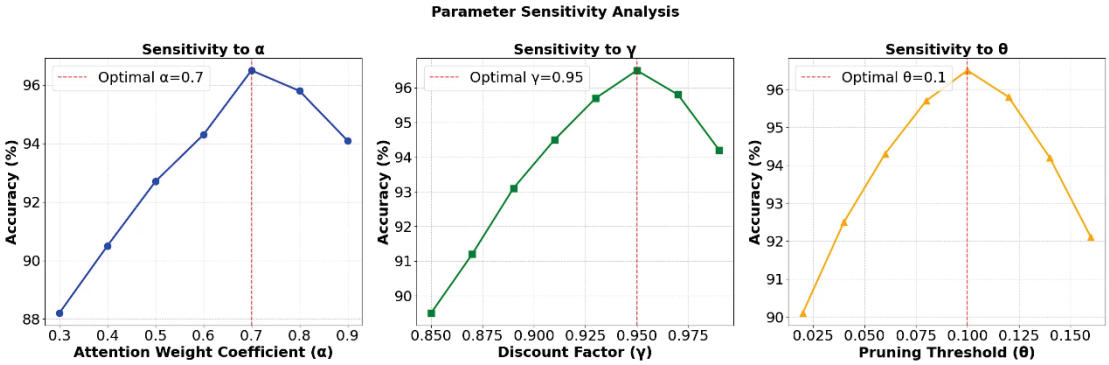


Figure 4. Parameter Sensitivity Analysis Curve.

In summary, the algorithm presented in this paper exhibits optimal localization performance in three scenarios: single root cause, multiple root cause, and dynamic link, balancing accuracy and search efficiency, and demonstrating good engineering applicability.

V. CONCLUSION

This study addresses the challenge of root cause localization for cross-layer link failures in microservices by proposing an intelligent localization algorithm incorporating deep reinforcement learning. The core research results are as follows: The constructed FPWDG model achieves unified representation of cross-layer fault features and quantification of dynamic propagation relationships, improving prediction accuracy by 28.6% compared to traditional static topology models; the designed Attention-DQN algorithm strengthens the feature weights of key nodes through an attention mechanism, and combined with a multi-objective reward function and pruning mechanism, effectively overcoming local optima limitations, achieving optimal performance in single root cause, multiple root cause, and dynamic link scenarios, with a maximum localization accuracy of 96.5% and an average troubleshooting step count of only 3.2 steps. The study has certain limitations: the experiments were conducted based on a simulated e-commerce system, and the performance in large-scale complex real-world business scenarios was not fully verified; the algorithm does not consider privacy protection requirements in multi-tenant

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 21:41:41 UTC from IEEE Xplore. Restrictions apply.

scenarios and has insufficient adaptability to new microservice architectures such as Serverless. Future research will focus on three aspects: first, developing a multi-tenant privacy-preserving location algorithm based on federated learning to achieve secure sharing of cross-tenant fault data; second, exploring early warning and root cause prediction technologies for faults by combining time-series prediction models to achieve proactive prevention; and third, optimizing algorithms to adapt to serverless architectures, improving location performance in stateless service scenarios, promoting the development of intelligent operation and maintenance technologies, and providing more comprehensive technical support for the stable operation of cloud-native systems.

ACKNOWLEDGMENTS

This work was supported by Science and Technology Project of NARI Group Co.Ltd of China: Research and Application of Key Technologies for Fault Localization in Microservice Architecture.

REFERENCES

[1] Zhang Qixun, Jia Tong, Yang Yong, & Li Ying. A method for root cause localization of microservice system faults based on service dependency graph. Frontiers of Data and Computing Development, vol. 6, pp. 87-95, April 2024.

[2] Tang Lun, Liao Hao, Cao Rui, Wang Zhaokun, & Chen Qianbin. A service function chain fault diagnosis algorithm based on deep dynamic Bayesian network. Journal of Electronics and Information Technology, vol. 43, pp. 3588-3596, December 2021.

[3] Song Ruiwang, Yang Yang, & Wan Luchang. Construction and practical exploration of intelligent operation and maintenance system for power fiber optic communication network. Smart City Applications, vol. 8, pp. 32-34, August 2025.

[4] Guo Bin, Liu Sicong, Liu Yan, Li Zhigang, Yu Zhiwen, & Zhou Xingshe. Intelligent Internet of Things: Concept, System Architecture and Key Technologies. Chinese Journal of Computers, vol. 46, pp. 2259-2278, November 2023.

[5] Zhang Lu, Liu Hui, Jiang Jiajun, Wang Bo, Li Qingyuan, Zhong Wenkang, ... & Luo Bin. Preface to a Special Topic on Defect Detection and Repair Technology for Complex Software. Journal of Software, vol. 35, pp. 3069-3070, July 2024.

[6] Li Yunxuan, Yang Yaping, Tu Jiayi, Gu Rentao, & Ji Yuefeng. Research on Key Technologies of Optical Networks for Interconnection between Intelligent Computing Centers. Telecommunications Science, vol. 41, pp. 3-19, April 2025.

[7] Razzaq, A., & Ghayyur, S. A. A systematic mapping study: The new age of software architecture from monolithic to microservice architecture—awareness and challenges. Computer Applications in Engineering Education, vol. 31, pp. 421-451, February 2023.

[8] Cerny, T., Abdelfattah, A. S., Yero, J., & Taibi, D. From static code analysis to visual models of microservice architecture. Cluster Computing, vol. 27, pp. 4145-4170, April 2024.

[9] Xu Jinghang, Yan Biying, Chen Feng, & Cao Yaqi. Dynamic fusion of emergency response plans based on domain knowledge graph. Computer Systems Applications, vol. 30, pp. 1-13, August 2021.

[10] Ming Cheng. Research on fault diagnosis and remote maintenance technology of secondary equipment in wind farms. Hydropower Technology, vol. 8, pp. 100-102, April 2025.

[11] Wang Yujie, Wu Guoxin, Huang Ji, Wang Kun, & Zhang Guolong. A review of research on micro/nano structures and preparation technology of scintillators. Journal of Electronic Measurement & Instrumentation, vol. 39, pp. 1-13, January 2025.

[12] Chen Yuanliang, Ma Fuchen, Zhou Yuanhang, Yan Zhen, Jiang Yu, & Sun Jiaguang. A review of research on dynamic testing techniques for distributed systems. Journal of Software, vol. 36, pp. 2964-3002, July 2024.

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 21:41:41 UTC from IEEE Xplore. Restrictions apply.